CSC 212: Data Structures and Abstractions

11: Analysis of Recursive Algorithms

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



From last lecture …

- Recursive methods on linked lists
  - solve one node, then trust the recursive call with the rest of the list
- The **call tree** is the cost
  - one call per node: a chain of $n + 1$ calls
- Today:
  - read the running time straight off the call tree, then use it on two classic algorithms: binary search and merge sort

The recursion tree

- Draw the call tree; write the work of each call **next to it**; then add up **level by level**

```
// copies A on every call
int sum_array(std::vector<int> A, int n) {
    if (n == 1) return A[0];
    int s = sum_array(A, n - 1);
    return A[n - 1] + s;
}
```

[draw the call tree and label the nodes with cost]

- **Draw the call tree** for the input $A = \{1, 3, 2, 5, 6\}$
  - every call copies **all** of A

$$T(n) = \underbrace{n + n + \cdots + n}_{n \text{ calls}} = n^2$$

The copies stay alive while their calls are pending: $\Theta(n^2)$ memory, not just time.

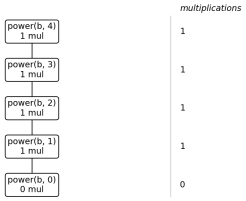
The recursion tree method

- Every call does some **own work**
  - everything except its recursive call(s)
- The total cost $T(n)$ is the sum of the own work of all calls
- Four steps to calculate $T(n)$:
  - draw the call tree
  - next to each call, write its own work
  - add up each level
  - add up the levels

A chain of calls

- Single recursive call: the tree is a chain of $\Theta(n)$ calls
- `power` example:
  - ✓ each level does 1 multiplication, the base case does none

```
double power(double b, int n) {
    if (n == 0)
        return 1;
    return b * power(b, n - 1);
}
```



4 levels of 1 multiplication: n multiplications, $\Theta(n)$

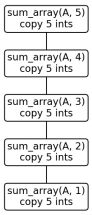
A chain with heavy nodes

- Same chain shape as `power`, but every node copies all n ints

```
int sum_array(std::vector<int> A, int n) {
    if (n == 1) return A[0];
    int s = sum_array(A, n - 1);
    return A[n - 1] + s;
}
```

$$\underbrace{n + n + \cdots + n}_{n \text{ levels}} = n^2 \quad \Rightarrow \quad \Theta(n^2)$$

passing by reference is about 540,000x faster at n = 20,000



n levels of n copies: $n \times n = n^2$ **copies,** $\Theta(n^2)$

Q11-1

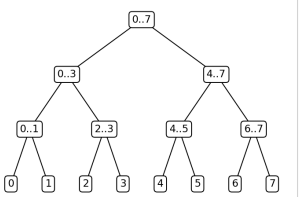
- Maximum of an array, by halves: find the max of each half, return the larger. What is the running time of `max_half` on n elements?

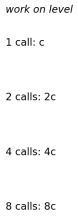
```
int max_half(const std::vector<int>& A, size_t lo, size_t hi) {
    if (lo == hi) return A[lo];          // one element
    size_t mid = lo + (hi - lo) / 2;
    int a = max_half(A, lo, mid);        // two calls
    int b = max_half(A, mid + 1, hi);
    return a > b ? a : b;
}
```

A. $\Theta(1)$
B. $\Theta(\log n)$
C. $\Theta(n)$
D. $\Theta(n \log n)$
E. None of the above

Answer

- C) $\Theta(n)$
  - ✓ the call tree for $n = 8$: each node is the range a call searches, and each call does c work besides its calls





$c(1 + 2 + 4 + \cdots + n) = c(2n - 1)$**:** $\Theta(n)$

Recurrences

- A **recurrence** defines a function in terms of its values on smaller inputs, along with a **base case**
- The running time of a recursive function is a recurrence

$$T(n) = \underbrace{\text{work in this call}}_{\text{not counting the calls}} + \underbrace{\text{cost of the recursive calls}}_{T \text{ on smaller inputs}}$$

- **Solving** a recurrence means finding a **closed form**: a formula for $T(n)$ without T on the right
  - for algorithms, a Θ answer is usually enough

a recurrence is the tree written as an equation

Solving recurrences

- Goal
  - determine the asymptotic running time $T(n)$ of a recursive algorithm
- Three common methods:
  - **recursion tree**: draw recursion tree, calculate the work at each level, and sum across levels.
  - **iteration / unrolling**: repeatedly expand the recurrence to identify a pattern and derive a bound
  - **master theorem**: apply standard cases to recurrences of the form $T(n) = aT(n/b) + f(n)$
  - and others …

Halving the exponent (better `power`)

- Compute the half power once, then square it
  - main idea: $b^8 = (b^4)^2$ and $b^9 = b \times (b^4)^2$

```
double power_2(double b, int n) {
    if (n == 0) return 1;
    double half = power_2(b, n / 2);     // one call, on n/2
    if (n % 2 == 0) return half * half;  // 1 multiplication
    else return b * half * half;         // 2 multiplications
}
```

$T(0) = 0, \quad T(n) \le T(n/2) + 2 \quad \Rightarrow \quad T(n) \le 2\log_2 n + 2, \quad \Theta(\log n)$

  - $b^{1{,}000{,}000}$: `power` makes 1,000,000 multiplications, `power_2` at most 40

[draw the call tree and label the nodes with cost]

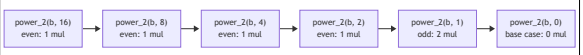
Q11-2

- How many multiplications does `power_2(b, 16)` do?

A. 4
B. 5
C. 6
D. 16
E. None of the above

Answer

- C) 6



Halving and log_2

- How many times can we halve n before we reach 1?

$$\frac{n}{2^k} \le 1 \iff n \le 2^k \iff \log_2 n \le k$$

  - ✓ the smallest such k is $\lceil \log_2 n \rceil$
  - ✓ example: $16 \to 8 \to 4 \to 2 \to 1$, four halvings, and $\log_2 16 = 4$

n	n steps	$\log_2$ n steps
10^9	0.0024 s	about 30
10^{12}	2.4 s	about 40
10^{15}	40 minutes	about 50
10^{18}	28 days	about 60
10^{20}	7.7 years	about 66

412,090 MIPS is the Intel Core i9-9900K's rating at 4.7 GHz. Times assume one instruction per step.

Shrink the problem by a factor, not by a constant, and the depth becomes logarithmic.

Q11-3

- A student writes the maximum of a non-empty list like this

```
int max_r(Node* p) {
    if (p->next == nullptr)
        return p->data;
    if (p->data > max_r(p->next))
        return p->data;
    return max_r(p->next);
}
```

- The list is *1 2 3 ... 30*, increasing. How many calls does `max_r(m_head)` make?
  - A. 30
  - B. 60
  - C. 100
  - D. About $2^{30} \approx 10^9$
  - E. None of the above

Answer

- D) About $2^{30} \approx 10^9$

$$T(1) = 1, \quad T(n) = 1 + 2\,T(n-1) \quad \Rightarrow \quad T(n) = 2^n - 1$$

```
int max_r(Node* p) {
    if (p->next == nullptr)
        return p->data;
    if (p->data > max_r(p->next))
        return p->data;
    return max_r(p->next);
}
```

[draw the call tree and label the nodes with cost]

Binary search (recursive)

Binary search

- On a **sorted array**: find whether *k* is present
  - ✓ base case: the input is empty, *k* is not there
  - ✓ compare *k* with the middle element
    - equal: found
    - smaller: **search recursively** the left half
    - larger: **search recursively** the right half

k = 48

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	5	10	15	20	22	30	35	40	43	48	51
lo												hi

```
call 1: lo = 0, hi = 12, mid = 6, A[6] = 22 < 48: go right
call 2: lo = 7, hi = 12, mid = 9, A[9] = 40 < 48: go right
call 3: lo = 10, hi = 12, mid = 11, A[11] = 48: found
```

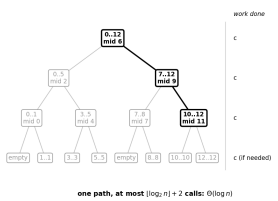

The code

```
bool b_s(const std::vector<int>& A, size_t lo, size_t hi, int k) {
    if (hi < lo) return false;          // empty range
    size_t mid = lo + (hi - lo) / 2;    // safer than (lo + hi) / 2
    if (A[mid] == k) return true;
    else if (A[mid] < k)
        return b_s(A, mid + 1, hi, k); // right half
    else {
        if (mid == 0) return false;     // mid - 1 would wrap around
        return b_s(A, lo, mid - 1, k); // left half
    }
}
```

- called as `bsearch(A, 0, A.size() - 1, k)` on a non-empty array
- `(lo + hi) / 2` can overflow for very large arrays
- `size_t` cannot hold `−1`: the `mid == 0` check avoids the wraparound

The call tree of binary search

- Each call makes at most one call: the call tree is a chain
- Draw every range binary search could visit: a tree of depth about $\log_2 n$
  - ✓ one search walks one path from the root down
  - ✓ *k = 48*: three calls, bold below



$$T(0) = c, \quad T(n) = T(n/2) + c \quad \Rightarrow \quad \Theta(\log n)$$

Q11-4

- How many calls does `b_s` make for $k = 3$, which is not in the array?

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	5	10	15	20	22	30	35	40	43	48	51

A. 1
B. 13
C. 4
D. 6
E. None of the above

Answer

- E) None of the above

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	5	10	15	20	22	30	35	40	43	48	51

- 5 calls

```
call 1: lo = 0, hi = 12, mid = 6, A[6] = 22 > 3: go left
call 2: lo = 0, hi = 5, mid = 2, A[2] = 5  > 3: go left
call 3: lo = 0, hi = 1, mid = 0, A[0] = 1  < 3: go right
call 4: lo = 1, hi = 1, mid = 1, A[1] = 2  < 3: go right
call 5: lo = 2, hi = 1: empty range, return false
```

Measured on 10^7 sorted ints: linear search 3.4 ms for a target near the end, binary search 0.23 microseconds, about 15,000x faster.

Merge sort

Sorting

- Given a sequence of n elements that can be compared according to a total order relation, rearrange them in monotonic order (non-decreasing or non-increasing)
- The output of any sorting algorithm must satisfy two conditions:
  - it is in **monotonic order**: each element is no smaller/larger than the previous one
  - it is a **permutation**: a reordering that retains all of the original elements

`central problem in computer science`

Merge sort

- **Divide** the array into two halves
  - calculate the midpoint to split the array
- **Conquer** each half recursively
  - call merge sort on each half (solve 2 smaller problems)
- **Combine** the solutions
  - after both recursive calls finish, **merge** the two sorted halves into one sorted array

Divide and Conquer Methods
A problem-solving approach that breaks a problem into smaller subproblems, solves them independently, and then combines their solutions. Examples: binary search, merge sort, quick sort.

Merge sort

0	1	2	3	4	5	6	7
4	2	1	10	6	20	7	30



Merge sort algorithm

```
if (hi <= lo) return

mid = lo + (hi - lo) / 2

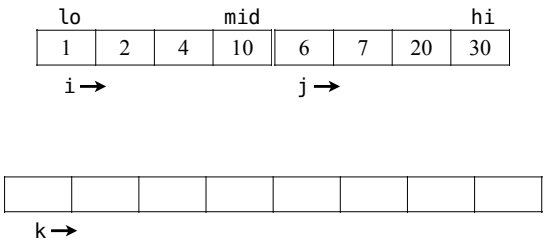
mergesort(A, lo, mid)
mergesort(A, mid+1, hi)

merge(A, lo, mid, hi)
```

lo			mid				hi
4	2	1	10	6	30	20	7

```
void merge_sort(std::vector<int>& A, size_t lo, size_t hi) {
    if (hi <= lo) return;
    size_t mid = lo + ((hi-lo) / 2);
    merge_sort(A, lo, mid);
    merge_sort(A, mid+1, hi);
    merge(A, lo, mid, hi);
}
```


Merging two sorted arrays



A temporary array is necessary to guarantee a linear time operation

Show me the code

```
void merge(std::vector<int>& A, size_t lo, size_t mid, size_t hi) {
    std::vector<int> B(hi - lo + 1);
    size_t i = lo, j = mid + 1, k = 0;

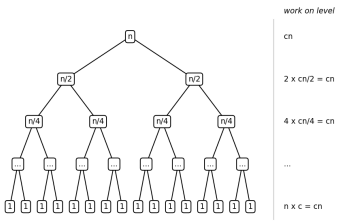
    while (i <= mid && j <= hi) {
        if (A[i] <= A[j]) {
            B[k++] = A[i++];
        } else {
            B[k++] = A[j++];
        }
    }
    while (i <= mid) B[k++] = A[i++];
    while (j <= hi) B[k++] = A[j++];

    for (k = 0 ; k < B.size() ; ++k) {
        A[lo + k] = B[k];
    }
}
```


The recursion tree of merge sort

$T(0) = 0$
$T(1) = 0$
$T(n) = 2T(n/2) + cn$

```
void merge_sort(std::vector<int>& A, size_t lo, size_t hi) {
    if (hi <= lo) return;
    size_t mid = lo + ((hi-lo) / 2);
    merge_sort(A, lo, mid);
    merge_sort(A, mid+1, hi);
    merge(A, lo, mid, hi);
}
```



$\log_2 n$ **levels that merge,** cn **each:** $cn \log_2 n$**,** $\Theta(n \log n)$

Insertion sort

- Start at index 1, loop through the array
  - ✓ compare with the element to its left
  - ✓ if smaller, swap them and move left
  - ✓ repeat until element is not smaller or you reach the start

Worst-case $\Theta(n^2)$**:** input is reverse sorted

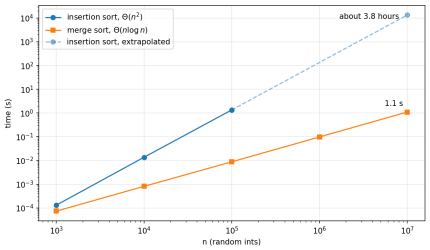
Best-case $\Theta(n)$**:** input is already or almost sorted

Average-case $\Theta(n^2)$**:** expect every element to move $O(n/2)$ times on average

```
void insertion_sort(std::vector<int>& A) {
    for (size_t i = 1 ; i < A.size() ; ++i) {
        for (size_t j = i; j > 0 ; --j) {
            if (A[j] < A[j-1]) {
                std::swap(A[j], A[j-1]);
            } else {
                break;
            }
        }
    }
}
```


Insertion sort vs merge sort

- $n = 10^7$: merge sort 1.1 s; insertion sort about 3.8 hours, about 12,000x slower



Intel Core i7-8700B, clang -O2, random ints from std::mt19937, best of 3 runs.
Insertion sort was measured up to 10^5 (1.36 s) and extrapolated by n^2 beyond that.

Final comments

- Major disadvantage
  - a sorting algorithm is in-place if it uses O(log n) extra memory
  - merge sort is NOT **in-place** as it requires O(n) auxiliary memory for the temporary array
- Improvements
  - use insertion sort for small arrays
  - stop recursion if subarray already sorted